



Compare web vs native SaaS onboarding conversion drop-offs, deferred signup patterns, and time-to-value milestones in a teardown report

Computer

Map the UX user journey from landing page to first aha moment across web and native SaaS apps with a comparative flow diagram

Computer

Show me the user journey architecture flowchart

Compare web and native conversion drop-offs

Map the deferred signup and token handoff flow

Semester should use **web-first acquisition and activation**, with **native apps** as a **continuity and capability layer**. The user must be able to discover Semester, understand it, create an account, complete a meaningful first action, and return—all without being forced into an app store or losing context.

The central UX rule is:

Do not make installation, long signup forms, or profile completion the price of seeing value. Make the first useful outcome easy; ask for identity, data, permissions, and installation only when each unlocks a clear next benefit.

Comparative teardown

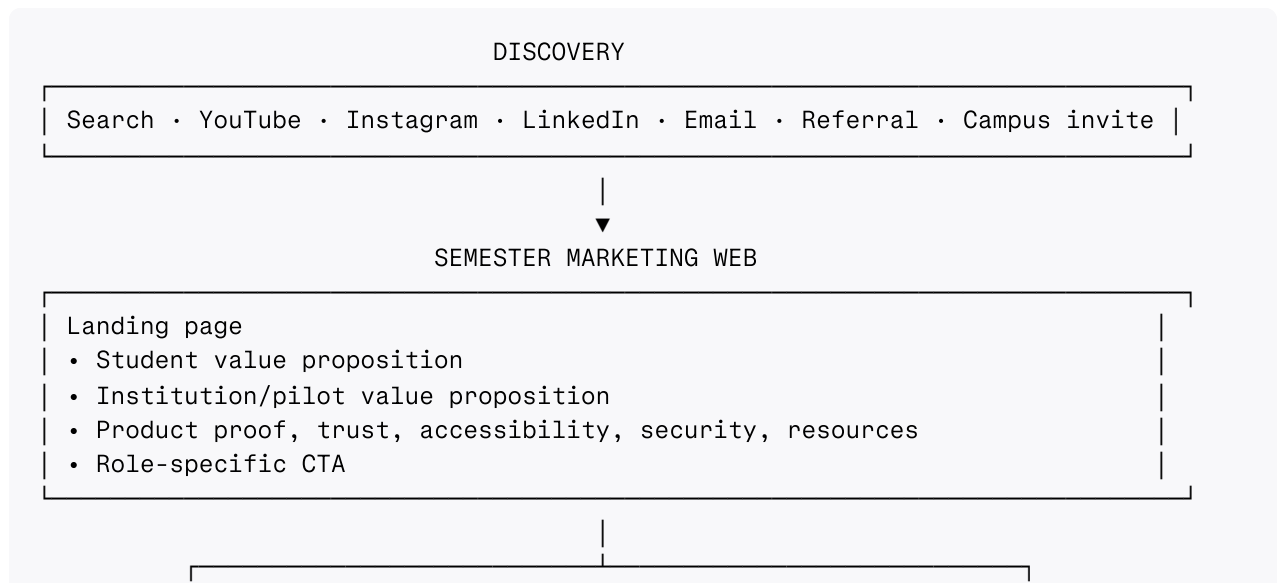
Web versus native onboarding

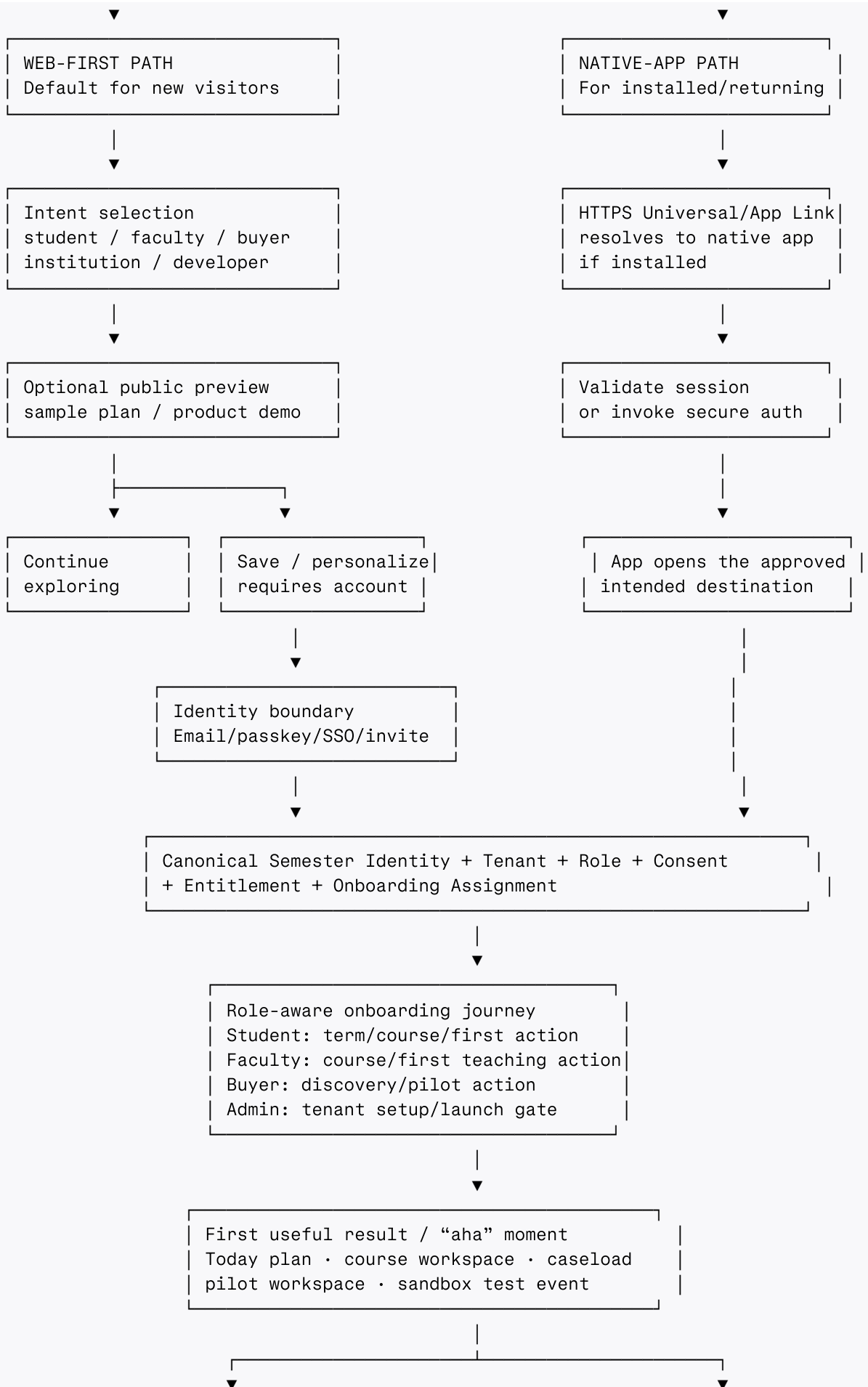
Dimension	Web / responsive app	Native mobile app	Semester architecture decision
Discovery	Excellent: search, social, email, campus links, YouTube, referrals, and SEO all open instantly	Weak for new prospects because the user must already have the app or install it	Use public web as the universal entry point
Marketing-to-product transition	Can preserve source, campaign, role intent, and route in one browser session	Can break at app store, device permission, or app-switch transitions	Complete early journey in browser
Signup	Desktop keyboard, password manager, passkeys, SSO, and browser recovery are typically simpler	Small screens, keyboard friction, app permission context, store interruption	Offer full signup in web and native, but do not force native

Dimension	Web / responsive app	Native mobile app	Semester architecture decision
Institutional SSO	Browser is the expected setting for federated/enterprise authentication	Can work via secure system browser, but in-app web views add risk/complexity	Use system browser for SSO on every platform
First value	Immediate if Semester's browser app is fully functional	Strong after app is installed and authenticated	Deliver first value in responsive web
Download/install barrier	None	Store listing, download time, sign-in again, deep-link loss	Never require download for a new user's first useful experience
Push notifications	Limited/inconsistent by browser/platform	Strong and reliable	Prompt after demonstrated value, not on launch
Offline support	Good with PWA for defined offline-safe experiences	Best for offline cache, camera, biometrics, device APIs	PWA first; native when capabilities justify it
Updates	Instant deployment	Store rollout and version fragmentation	Treat web as universal baseline
Deep link reliability	Strong: an HTTPS URL routes directly to page	Requires Universal Links/App Links configuration and robust fallback	Every native route needs a matching web route
Returning-user continuity	Strong if session persists	Strong if account and state synchronize server-side	Use one identity and onboarding state across all clients
Main drop-off risk	Long form, account verification, empty first screen	Install/app-store handoff, app launch context loss, permission prompts	Focus optimization effort on verification, first value, and state restoration

Native apps should improve an already useful experience—not become a gate between marketing and value.

Comparative user journey diagram





Continue in responsive web
immediately

Optional PWA/native
install, preserving path

Conversion drop-offs

Web-first funnel risks

Step	User concern	Common failure	Semester response
Landing	“Is this for me?”	Platform message is too broad	Show student and institution paths immediately
CTA	“What happens next?”	CTA leads to a generic form	Label outcome: “Build your plan,” “Continue with campus,” “Design a pilot”
Preview	“Is this useful?”	User sees empty product or marketing demo only	Let them generate a safe, non-sensitive preview result
Signup	“Why do you need all this?”	Long forms and early profile demands	Request only identity + minimum intent
Verification	“I lost my progress”	Email link opens a blank app home	Preserve encrypted temporary draft and approved destination
Onboarding	“This feels like work”	Long feature tour, no immediate result	Make one meaningful outcome the only early goal
First app screen	“Now what?”	Empty dashboard or dense menu	Generate Today/next action based on known intent
Return	“Why come back?”	No reminder or incomplete plan is invisible	Let user set a useful reminder after value is created

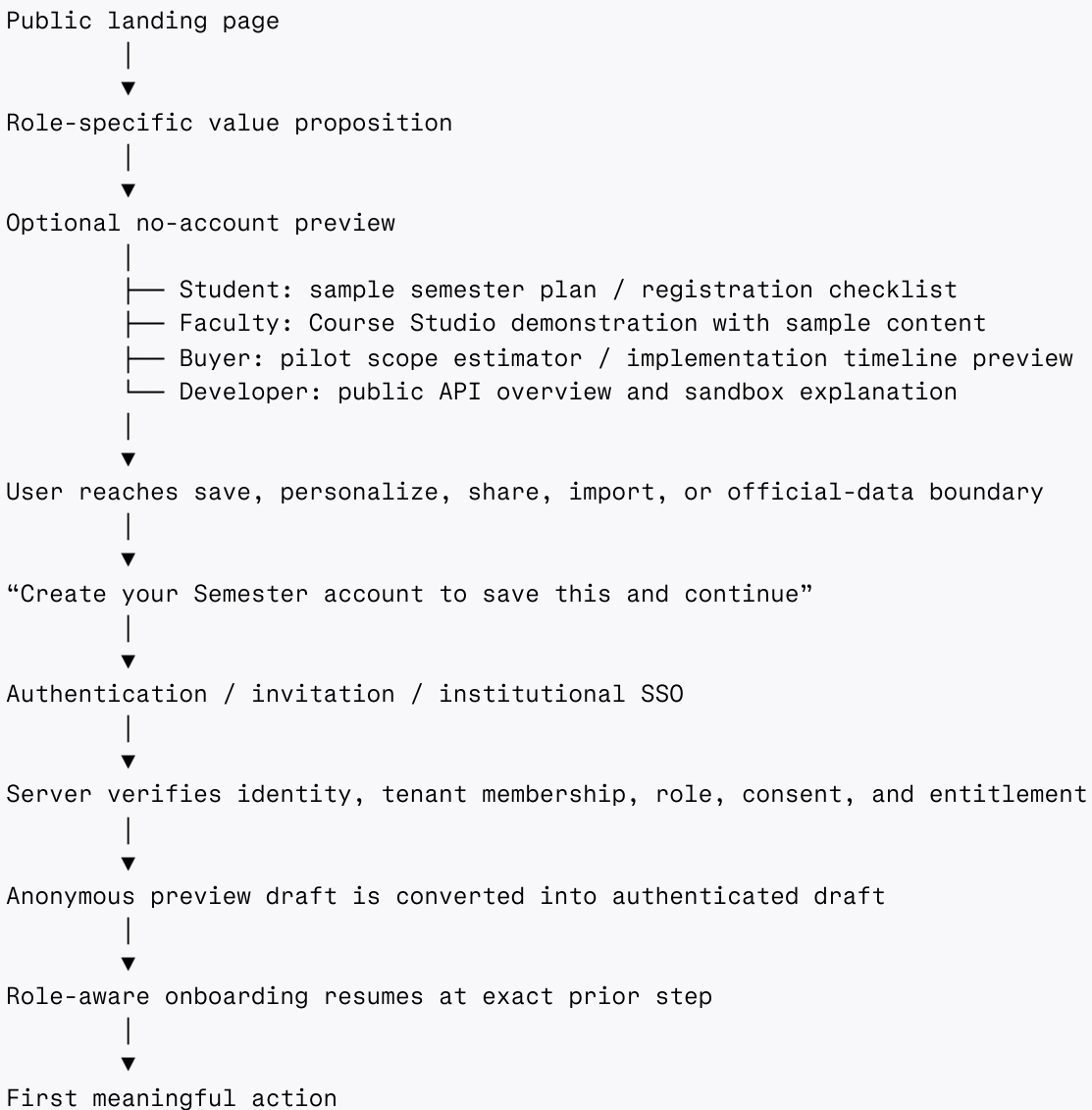
Native-first funnel risks

Step	User concern	Common failure	Semester response
Store redirect	“Do I really need another app?”	User abandons before installation	Never make install mandatory for initial value
App store	“What is this / is it trustworthy?”	Listing differs from website promise	Keep store assets, screenshots, terms, and brand consistent
Download	“This is taking too long”	User leaves during install	Keep browser fallback and send no user to a dead end
First open	“Where is the thing I clicked?”	App opens generic home screen	Restore the approved route and onboarding checkpoint
Authentication	“Why must I sign in again?”	Web session does not hand off securely	Use secure browser-based auth and server-issued native session
Permissions	“Why are you asking for access?”	Push/camera/calendar requests on launch	Ask immediately before a user-triggered benefit

Step	User concern	Common failure	Semester response
App updates	"Why is this broken?"	Version mismatch or forced upgrade	Feature-gate by app version; provide web fallback
Native navigation	"How do I get back?"	Deep links have no clear exit/back behavior	Define back-stack and web-return behavior per link type

Deferred signup pattern

Deferred signup should allow a user to experience a narrow, safe preview before creating an account. It does **not** mean anonymous access to personal, institutional, financial, or administrative data.



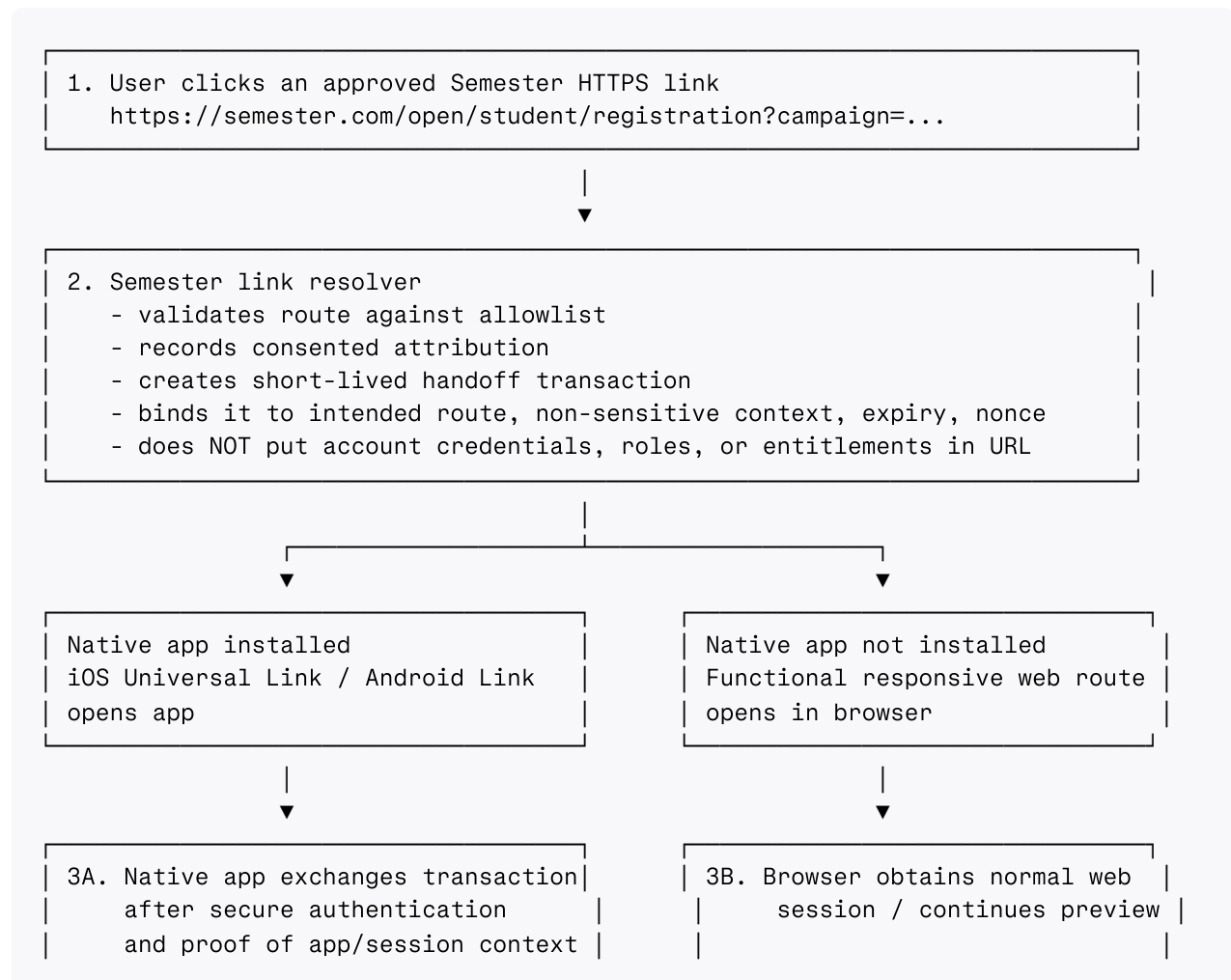
Deferred-signup eligibility

Can be deferred	Must require authenticated account
Reading public product pages	Saving across devices
Browsing public resources	Importing calendar/course data
Trying an anonymous sample planning tool	Viewing official institution data
Choosing a broad goal	Viewing grades, holds, degree audit, billing, or student records
Viewing a publicly available product walkthrough	Support requests containing personal information
Building a temporary sample schedule	Payment, subscription, or refund actions
Viewing public pricing	Community posting or peer contact
Reading public trust materials	Faculty/admin/operator actions

Token and handoff flow

Do not hand an authenticated browser access token directly through an app-store link, query string, URL fragment, QR code, or marketing platform. Tokens can leak through logs, browser history, referer headers, screenshots, shared devices, and analytics.

Use a short-lived **handoff transaction** instead.



-
- ```
graph TD; A[] --> B[4. Semester server checks]; B --> C[5. Route user];
```
4. Semester server checks
- authenticated account
  - membership and tenant
  - role/capability
  - consent and entitlement
  - destination authorization
  - onboarding assignment/progress

5. Route user
- onboarding incomplete: exact required next step
  - onboarding complete: exact approved in-app destination
  - unauthorized: safe explanation + support/recovery route

## Handoff transaction model

```
type HandoffTransaction = {
 id: string;
 nonce: string;

 destination: {
 route:
 | "student_today"
 | "student_plan"
 | "student_registration"
 | "course_home"
 | "advisor_caseload"
 | "institution_setup"
 | "pilot_workspace"
 | "developer_sandbox";
 resourceId?: string;
 tenantHint?: string;
 };

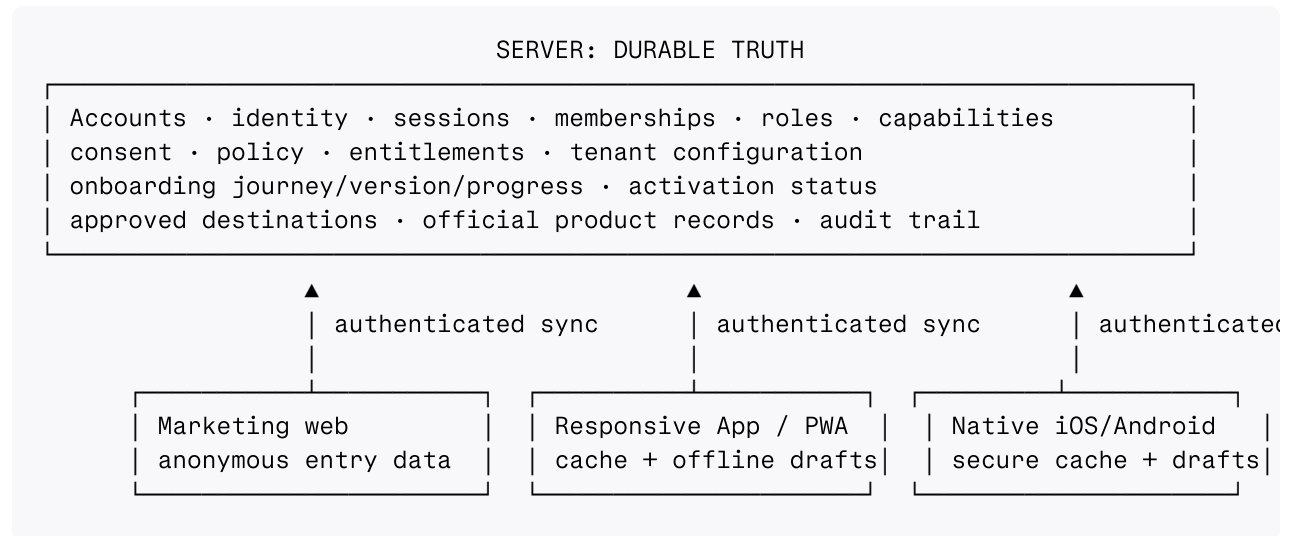
 entryContext: {
 source?: string;
 campaignId?: string;
 referralCode?: string;
 intendedRole?: string;
 };

 expiresAt: string;
 consumedAt?: string;
 status: "created" | "consumed" | "expired" | "revoked";
};
```

#### Important:

- The transaction is not an access token.
- It cannot grant identity, tenant membership, role, entitlement, or capability.
- It expires quickly and is single-use.
- The native app/browser exchanges it only after normal authentication.
- The server independently authorizes the final route.
- Only non-sensitive campaign and intent context travels through a link.

## State persistence model



## What persists where

| State                          | System of record                  | Browser                 | PWA                            | Native app                            | Security rule                                 |
|--------------------------------|-----------------------------------|-------------------------|--------------------------------|---------------------------------------|-----------------------------------------------|
| Account, membership, role      | Semester server                   | Scoped session only     | Scoped session only            | Scoped session only                   | Never trust client role claims                |
| Consent and policy acceptance  | Semester server                   | Cached display state    | Cached display state           | Cached display state                  | Server validates current version              |
| Onboarding assignment/progress | Semester server                   | Resume checkpoint cache | Resume checkpoint cache        | Resume checkpoint cache               | Server versions and resolves status           |
| Intent / continue destination  | Server transaction                | Short-lived reference   | Short-lived reference          | Short-lived reference                 | Allowlist + expiration + one-time use         |
| Anonymous preview draft        | Browser/session storage initially | Local temporary draft   | Optional local temporary draft | Usually not applicable before install | Expire quickly; do not use for sensitive data |
| Authenticated product draft    | Server plus offline queue         | Short-lived local draft | Encrypted local offline draft  | Secure encrypted draft                | Conflict handling + idempotency               |

| State          | System of record               | Browser                   | PWA                     | Native app              | Security rule                               |
|----------------|--------------------------------|---------------------------|-------------------------|-------------------------|---------------------------------------------|
| Official data  | Semester/institution authority | Read cache only           | Read cache only         | Read cache only         | RLS/capability checks on every fetch        |
| UI preferences | User profile server-side       | Local cache               | Local cache             | Local cache             | Do not mix with authorization               |
| Device state   | Device registry server-side    | Browser/device identifier | Installation identifier | Installation identifier | Revocable and auditable                     |
| Push tokens    | Server-device association      | Browser push endpoint     | Browser push endpoint   | Native push token       | Permission and lifecycle managed separately |

## Time-to-value milestones

### Student

0:00 – lands on student page  
 0:15 – sees a specific promise: organize the next academic action  
 0:45 – chooses a goal or tries safe preview  
 1:30 – creates/verifies account only at save/personalize boundary  
 2:30 – selects term or begins registration checklist  
 3:30 – receives a personalized Today view  
 4:00 – completes first meaningful action  
 After value – receives optional reminder, PWA install, or notification prompt

**Student activation** is not signup. It is one of:

- A real term/course/plan created.
- A registration-readiness task completed.
- A real academic task completed or scheduled.
- A relevant official next action opened and acknowledged, when institution data is available.

### Faculty

0:00 – arrives via institution route or invitation  
 0:30 – completes institution SSO  
 1:00 – selects assigned course  
 1:30 – chooses intent: prepare course, add resource, create assessment, review work, or set office hours  
 2:30 – completes first teaching action  
 3:00 – sees Course Studio workspace with next recommended action

**Faculty activation** is a real course operation: publish resource, configure course component, create assessment, establish office hours, or take a supported teaching action.

## Advisor

```
0:00 – SSO/invite enters advisor workspace
0:30 – confirms caseload and notification preference
1:00 – sees prioritized student work queue
2:00 – opens first student action
3:00 – completes check-in, appointment, referral, or success-plan step
```

**Advisor activation** is completion of a governed student-support action, not merely viewing a dashboard.

## Institutional admin

```
0:00 – receives invitation / SSO
1:00 – confirms tenant/admin authority
2:00 – sees implementation checklist
3:00 – completes configuration, identity, module, or policy action
5:00+ – sees launch gate and next accountable owner
```

**Admin activation** is a governed configuration or implementation milestone completed with the right authorization and evidence.

## Native routing requirements

Use normal HTTPS routes as the canonical representation of content and actions:

```
https://semester.com/app/t/{tenantSlug}/student/today
https://semester.com/app/t/{tenantSlug}/student/registration
https://semester.com/app/t/{tenantSlug}/course/{courseId}
https://semester.com/app/t/{tenantSlug}/advisor/caseload
https://semester.com/app/t/{tenantSlug}/admin/implementation
```

For iOS, Universal Links depend on an association between the app and website; if the app is not installed, the same website can serve the content in a browser. Android App Links similarly use verified HTTPS associations and can send users to the associated app while retaining a normal web fallback when the app is absent. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

## Release requirements

### Marketing and conversion

- [ ] Each audience has a clear primary CTA.
- [ ] Every CTA preserves entry context without exposing sensitive data.
- [ ] Campaign pages have an explicit conversion event.
- [ ] Claims, logos, customer outcomes, and screenshots are truthful and approved.

### Identity and account creation

- [ ] Browser and native clients use one canonical account system.
- [ ] Email/passkey/SSO/invite paths are tested.
- [ ] Account linking prevents accidental duplicates.

- [ ] Verification preserves destination and temporary safe draft state.
- [ ] Password-manager, copy/paste, and accessible-authentication behavior work.
- [ ] Recovery, logout, session revocation, and support routes work.

#### Onboarding and activation

- [ ] Every role has a defined activation event.
- [ ] The first experience has a real result, not an empty dashboard.
- [ ] Progress is resumable across browser, PWA, and native clients.
- [ ] Required policy/consent steps are versioned and auditable.
- [ ] Optional profile data is deferred until valuable.
- [ ] Support handoff works from every onboarding state.

#### Web-to-native handoff

- [ ] Every app destination has an equivalent browser route.
- [ ] Universal Links/App Links have verified production configuration.
- [ ] Installed, not-installed, expired-session, unauthenticated, unauthorized, old-app-version, and offline states are tested.
- [ ] Handoff transactions expire, are single-use, and do not contain credentials.
- [ ] Native app restores approved destination after authentication.
- [ ] Browser fallback is always usable.

#### Measurement

- [ ] Funnel events exclude sensitive academic, financial, identity, and support data.
- [ ] Metrics distinguish signup, verification, onboarding completion, first value, activation, and retention.
- [ ] Drop-off reports are segmented by role, institution policy, platform, entry source, and accessibility-safe categories.
- [ ] Experiments use explicit cohort controls and rollback.

The final design choice is straightforward: use the web to convert and activate, use native apps to deepen daily use, and let one secure server-authoritative identity and onboarding system preserve the journey everywhere.

✱

1. <https://developer.apple.com/documentation/xcode/supporting-universal-links-in-your-app>
2. <https://developer.apple.com/documentation/xcode/supporting-associated-domains>
3. <https://developer.android.com/training/app-links/about>
4. <https://developer.android.com/training/app-links>
5. <https://www.avanderlee.com/swiftui/universal-links-ios/>
6. <https://app.smler.io/blogs/deep-linking/app-links-vs-universal-links-technical-comparison-guide-2026>
7. <https://ptkd.com/journal/android-app-links-verification-assetlinks>
8. <https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum.html>
9. <https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/>
10. <https://www.w3.org/TR/WCAG22/>
11. <https://help.branch.io/developer-hub/docs/android-app-links>
12. <https://idura.eu/blog/universal-links-and-app-links>
13. <https://www.appsflyer.com/glossary/universal-links/>

14. <https://www.airbridge.io/tips/blog/universal-links-android-app-links-deep-linking-guide>

15. <https://appfollow.io/glossary/android-app-links>